CoRT: Code-integrated Reasoning within Thinking

Taro Yamada[1]

[1]Anonymous Institution

Abstract

Large Reasoning Models (LRMs) achieve impressive performance through extended chain-of-thought reasoning, yet remain brittle and token-inefficient when precise mathematical computation is required. We introduce **CoRT** (**Co**de-integrated **R**easoning within **T**hinking), a framework that teaches LRMs to strategically interleave code execution within their reasoning traces. Our key insight is that carefully designed textual "hints"—minimal natural-language cues inserted at critical reasoning junctures—can reliably trigger code generation at the right moments, raising code-invocation rates from roughly 50% to over 90%. Starting from only 30 curated exemplars, we synthesize high-quality training data and apply a progressive post-training pipeline of supervised fine-tuning, rejection fine-tuning, and reinforcement learning with code-execution rewards and output masking. Across five challenging mathematical benchmarks, CoRT yields consistent accuracy gains—up to +4 percentage points at 32B scale and +8 percentage points at 1.5B scale—while reducing token usage by 30–50%. Notably, our 32B model matches QwQ-32B accuracy on AIME 2024 using roughly half as many tokens. Behavioral analysis reveals a qualitative shift from verification-heavy to balanced calculation-and-verification code use, indicating that models learn *when* and *why* to invoke computation rather than applying it indiscriminately.

1 Introduction

Large Reasoning Models (LRMs) such as OpenAI o1 [1] and DeepSeek-R1 [2] have demonstrated remarkable mathematical reasoning capabilities through extended chain-of-thought (CoT) processes [3]. By "thinking longer," these models tackle competition-level problems that were previously out of reach. However, extended reasoning comes at a cost: LRMs frequently spend thousands of tokens on arithmetic computations that a simple program could execute in milliseconds, and such manual calculations remain a persistent source of errors [4].

A natural remedy is to integrate Code Interpreters (CIs) into the reasoning loop, allowing models to offload precise computation to deterministic execution environments. Prior work on tool-augmented reasoning—including PAL [5], Program of Thoughts [6], and ToRA [7]—has demonstrated the benefits of program-aided problem solving. More recently, approaches such as ToRL [8], ReTool [9], and START [10] have explored code integration specifically for LRMs, training models to produce interleaved text-and-code reasoning traces. Despite this progress, three fundamental challenges remain: (i) models struggle to decide *when* to invoke code versus continue reasoning in natural language; (ii) obtaining high-quality training data with naturally interleaved code-and-reasoning traces is difficult, as most existing datasets contain either pure text or pure code solutions; and (iii) standard reinforcement learning objectives do not explicitly reward efficient tool use.

In this paper, we introduce **CoRT** (**Co**de-integrated **R**easoning within **T**hinking), a framework that addresses all three challenges through a combination of *hint engineering* and a progressive post-training pipeline. Our central observation is that LRMs already possess latent code-generation abilities but lack the behavioral patterns to deploy them strategically during reasoning. Rather than

Preprint.

redesigning model architectures or collecting massive datasets, we show that minimal, carefully placed natural-language hints—as few as one or two sentences inserted at critical junctures in a reasoning trace—can reliably trigger appropriate code invocation. For instance, a cold-start hint such as "*let's solve this step by step using Python*" raises CI-triggering rates from approximately 50% to over 90%, while targeted hint-engineering cues discourage wasteful patterns such as re-verifying code-interpreter outputs in natural language.

Building on this insight, CoRT contributes a data-efficient synthesis pipeline that produces high-quality code-integrated reasoning traces from just 30 curated seed exemplars and 820 problems drawn from the STILL-3 dataset [11], yielding approximately 800 SFT and 830 RFT training examples. We then apply a progressive post-training pipeline—supervised fine-tuning (SFT), rejection fine-tuning (RFT) [12], and reinforcement learning (RL) with GRPO—incorporating several design choices tailored to code-integrated reasoning: persistent execution environments that maintain state across code calls within a single reasoning trace, output masking that prevents the model from hallucinating execution results, and a composite reward signal $R = R_a + \omega R_c$ that combines answer correctness with code-execution quality.

We evaluate CoRT on five challenging mathematical benchmarks: AIME 2024, AIME 2025, AMC 2023, MATH-500 [13], and OlympiadBench [14]. Our results demonstrate substantial gains at both 32B and 1.5B model scales. At 32B, CoRT improves accuracy by an average of 4 percentage points over the DeepSeek-R1-Distill-Qwen-32B baseline, with our best configuration (Prompt-Hint-SFT-32B) achieving 81.8% average accuracy across all benchmarks. Our Hint-Engineering-RFT-32B model matches QwQ-32B [15] on AIME 2024 while using roughly 50% fewer tokens (7K vs. 14K). At the 1.5B scale, RL proves particularly effective: Prompt-Hint-1.5B-RL achieves 58.3% average accuracy, a +8 percentage point improvement over SFT that surpasses DeepScaleR-1.5B-Preview [16]. Beyond accuracy, we observe a qualitative behavioral shift: models trained with CoRT transition from predominantly verification-oriented code usage to a balanced mix of computation and verification, indicating they learn not just *how* but *when* to invoke code.

Our contributions are as follows:

- A **hint-engineering data synthesis framework** that overcomes code-integrated reasoning data scarcity using only 30 high-quality seed exemplars and targeted natural-language cues.
- An **efficient post-training pipeline** (SFT $\rightarrow$ RFT $\rightarrow$ RL) with CI-aware rollout infrastructure, persistent environments, output masking, and code-execution rewards.
- **Strong empirical results** across five challenging benchmarks, with consistent accuracy gains and 30–50% token savings at both 32B and 1.5B scales.
- A **behavioral analysis** revealing a shift from verification-heavy to balanced code usage, providing insight into how models learn to integrate tools effectively.

2 Related Work

Reasoning in Large Language Models. Chain-of-thought prompting [3] demonstrated that eliciting step-by-step reasoning substantially improves LLM performance on complex tasks. This insight has been scaled in multiple directions: OpenAI o1 [1] and DeepSeek-R1 [2] train models to produce extended internal reasoning traces, achieving strong results on competition mathematics. QwQ-32B [15] showed that open-weight models can reach comparable performance through careful training. Complementary work has explored data-efficient approaches to reasoning: LIMO [17] demonstrated that a small number of carefully curated examples can elicit sophisticated reasoning, while s1 [18] and STILL-3 [11] investigated scaling test-time compute and difficulty-targeted training. DeepScaleR [16] further showed that RL-based post-training can unlock strong reasoning in small models. These methods, however, rely entirely on natural-language reasoning and remain susceptible to arithmetic errors in long computation chains.

Tool-Augmented Reasoning. Integrating external tools into language model reasoning has a rich history. Toolformer [19] trained models to decide when and how to call APIs, while PAL [5] and Program of Thoughts [6] delegated mathematical computation entirely to code execution. ToRA [7] combined natural-language reasoning with programmatic problem solving through interleaved tool-use trajectories. More recently, the focus has shifted to tool integration within LRMs: ToRL [8]

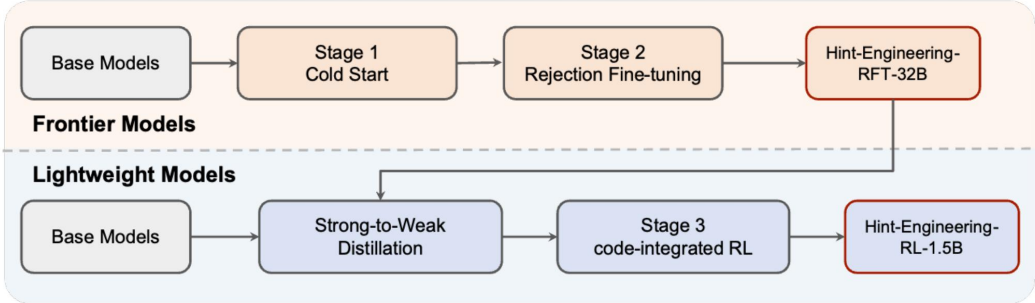


Figure 1: The CoRT training framework. Starting from a small set of curated exemplars, we construct hint-engineered traces and train the model through a three-stage pipeline: cold-start SFT, rejection fine-tuning (RFT), and GRPO-based reinforcement learning with code execution rewards.

applied RL to learn tool-calling policies, ReTool [9] explored code integration during extended thinking, and START [10] introduced structured tool-augmented reasoning training. Qu et al. [20] provided a broader survey of tool-integrated reasoning paradigms. Dong et al. [21] examined scaling laws for agentic tool use. While these approaches demonstrate the value of code integration, they typically require large-scale data collection or complex pipeline engineering. CoRT differs by showing that *minimal hint engineering*—targeted natural-language cues inserted into reasoning traces—can serve as an efficient bootstrap for code-integrated reasoning, dramatically reducing data requirements.

Post-Training for Reasoning. Reinforcement learning from outcome-based rewards has emerged as a powerful paradigm for improving reasoning. DeepSeekMath [22] introduced GRPO for mathematical reasoning, and subsequent work such as DAPO [23] refined the training dynamics. Rejection fine-tuning (RFT) [12] provides a simpler alternative that filters model-generated solutions by correctness. Yuan et al. [24] studied scaling properties of these approaches. CoRT builds on this foundation but introduces code-execution-aware modifications: persistent interpreter environments across rollout steps, masking of execution outputs to prevent hallucination, and a composite reward that explicitly credits successful code use. These adaptations are critical for stable training when reasoning traces contain interleaved text and executable code.

3 Method

In this section, we present the CoRT framework in detail. We first formulate the problem and describe the CoRT reasoning loop (Section 3.1), then detail the hint design strategies that guide tool use (Section 3.2), the data synthesis pipeline (Section 3.3), and the full post-training procedure (Section 3.4). An overview of the training framework is shown in Figure 1.

3.1 Problem Formulation and CoRT Framework

Given a mathematical problem x, a standard LRM generates a long-form reasoning trace r followed by a final answer a. CoRT extends this paradigm by integrating a persistent Python code interpreter (CI) into the reasoning process, formulating reasoning as iterative text–code–execution loops. Concretely, the model generates interleaved segments of natural-language reasoning t_i, code blocks c_i, and execution results e_i:

$$r = \big(t_1, c_1, e_1, t_2, c_2, e_2, \ldots, t_k, c_k, e_k, t_{k+1}\big), \tag{1}$$

where t_i represents a text reasoning segment, c_i is a Python code block, and e_i is the execution output returned by the CI. The model alternates between reasoning in natural language and offloading precise computations to the code interpreter, then incorporates the execution results back into its reasoning chain before producing the final answer a.

A key design choice is that the code interpreter maintains a *persistent execution environment* across all tool calls within a single reasoning trace. Variables, functions, and imported libraries defined in earlier code blocks remain available in subsequent ones, enabling the model to build upon prior computations incrementally rather than re-defining state at each step. We allow a maximum of 15

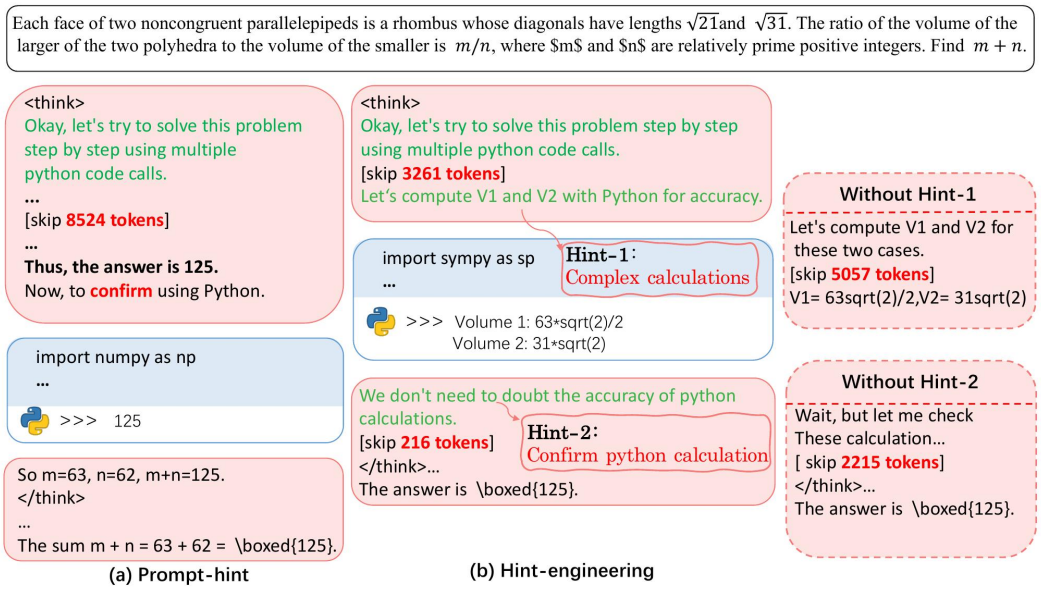


Figure 2: Comparison between prompt-hint and hint-engineering on AIME 2023 Problem 13. Prompt-hint inserts a single instruction at the start, broadly increasing code usage. Hint-engineering inserts targeted cues at specific moments, guiding the model to invoke code at computationally critical points and trust CI outputs.

tool calls per reasoning trace to balance between sufficient computational support and generation efficiency.

3.2 Hint Design

A central challenge in training code-integrated reasoning is that base LRMs do not spontaneously invoke tools at appropriate moments. We address this through two complementary *hint* strategies that guide the model toward effective tool use patterns.

Prompt-Hint. The simplest approach is a cold-start prompt hint: a fixed instruction inserted immediately after the `<think>` token at the beginning of the reasoning trace. Specifically, we prepend the sentence: *"Okay, let's try to solve this problem step by step using multiple python code calls."* This lightweight intervention boosts the code-triggering rate from approximately 50% to over 90%, providing a strong initialization signal for code-integrated reasoning [25, 26].

Hint-Engineering. While prompt-hints effectively increase overall code usage, they do not guide *when* and *how* the model should invoke tools within the reasoning trace. Hint-engineering addresses this by inserting targeted cues at specific moments during reasoning. We identify two key behavioral patterns that benefit from guidance:

- **Triggering computation early.** When the model begins manual arithmetic or symbolic manipulation, we insert the cue: *"It looks tedious, and we can use python code."* This redirects the model to offload computation to the CI before accumulating errors through lengthy manual derivations.
- **Trusting CI outputs.** Models often waste tokens re-verifying code interpreter results through manual recalculation. We insert the cue: *"We don't need to doubt the accuracy of python calculations."* This discourages redundant verification and encourages the model to proceed with the CI output, substantially reducing token usage.

Figure 2 illustrates the difference between prompt-hint and hint-engineering on a representative AIME problem. Hint-engineering produces more structured traces where code is invoked at computationally critical junctures, while prompt-hint leads to broader but less targeted code usage.

3.3 Data Synthesis

Training code-integrated reasoning requires high-quality traces demonstrating effective text–code–execution patterns, yet such data is scarce. We construct training data through a two-stage synthesis pipeline.

Cold-Start Exemplars. We manually curate 30 high-quality reasoning traces from AIME competition problems, applying hint-engineering to produce exemplars that demonstrate ideal code invocation patterns. These traces serve as few-shot demonstrations for the subsequent data generation process, following the cold-start philosophy of prior work [25, 26].

SFT Data. Using the 30 curated traces as exemplars, we generate hint-engineered reasoning traces for 820 problems from the STILL-3 dataset [11]. After filtering for correctness and trace quality, we obtain **800 SFT examples** that form the cold-start supervised fine-tuning set.

RFT Data. We apply rejection fine-tuning (RFT) [12, 24] to expand the training set. Using the SFT-trained model, we sample multiple candidate solutions for each problem and retain those that produce correct final answers. This process yields **830 RFT examples**, which are used to further refine the model before reinforcement learning.

Distillation. For the 1.5B model variant, we distill from a 32B teacher model. The 32B model first generates high-quality code-integrated reasoning traces, which are then used to train the smaller 1.5B student model through the same SFT $\rightarrow$ RFT pipeline.

3.4 Post-Training Pipeline

CoRT employs a three-stage post-training pipeline: cold-start SFT, rejection fine-tuning, and reinforcement learning.

Stage 1: Cold-Start SFT. We fine-tune the base LRM on the 800 hint-engineered SFT examples. This stage initializes the model's ability to generate interleaved text-code reasoning traces and establishes basic tool-use patterns.

Stage 2: Rejection Fine-Tuning. The SFT model is further trained on 830 RFT examples generated through rejection sampling [12, 24]. RFT reinforces successful reasoning strategies discovered by the model itself, improving robustness beyond what supervised examples alone provide.

Stage 3: Reinforcement Learning with GRPO. We apply Group Relative Policy Optimization (GRPO) [22] to further improve the model's reasoning quality. Following prior work [2, 16], we design a combined reward function:

$$R = R_a + \omega \cdot R_c, \tag{2}$$

where R_a is the *accuracy reward* that evaluates the correctness of the final answer, R_c is the *code execution reward* that measures whether the generated code runs successfully and produces useful outputs, and $\omega = 0.1$ controls the relative weight of the code reward. The accuracy reward provides the primary learning signal, while the code reward encourages the model to produce well-formed, executable code rather than syntactically broken or trivial snippets.

Two important implementation details ensure training stability with code-integrated rollouts. First, we apply **output masking**: the code execution outputs e_i are masked during loss computation, so the model does not backpropagate through tokens it did not generate (i.e., the CI outputs). This prevents the model from being penalized or rewarded for the verbosity or formatting of execution results, focusing the training signal on the model's own generation decisions. Second, the **persistent code execution environment** is maintained during RL rollouts, so that variables and state carry across multiple tool calls within a single episode, mirroring the inference-time setup. Each rollout permits up to 15 tool calls, matching the inference budget.

4 Experiments

4.1 Experimental Setup

We evaluate CoRT on five challenging mathematical reasoning benchmarks: AIME24, AIME25, AMC23, MATH500 [13], and OlympiadBench [14]. AIME and AMC benchmarks test competition-level problem solving, while MATH500 covers a broad range of difficulty levels and OlympiadBench targets olympiad-grade problems. We build on two base models: DeepSeek-R1-Distill-Qwen-32B and DeepSeek-R1-Distill-Qwen-1.5B [2], representing frontier and lightweight settings respectively. During inference, we set temperature to 0.6 and top_p to 0.95 following prior work [2]. Results on AIME24, AIME25, and AMC23 are averaged over 16 samples, while MATH500 and OlympiadBench results are averaged over 4 samples, to reduce variance. All experiments use a maximum sequence length of 32,768 tokens and limit tool usage to 15 calls per rollout.

We compare against state-of-the-art closed and open models including o1 [1], DeepSeek-R1 [2], and QwQ-32B [15], as well as recent tool-integrated baselines: START [10], STILL-3-TOOL [11], ReTool [9], and ToRL [8]. We also include DeepScaleR-1.5B-Preview [16] as a strong non-tool baseline in the lightweight setting.

4.2 Main Results

Table 1 presents the main results across all benchmarks. We organize our comparison into three tiers: SOTA proprietary/full-scale models, frontier 32B models, and lightweight 1.5B models.

Table 1: Performance comparison of different math reasoning models across benchmarks. For each section, best results are shown in **bold** and second-best results are underlined. During inference, we set temperature 0.6 and top_p 0.95. Results for AIME24, AIME25, and AMC23 are averaged over 16 samples, while MATH500 and Olympiad results are averaged over 4 samples. All experiments use a maximum sequence length of 32,768 tokens and limit tool usage to 15 calls.

Model	Tool-Use	AIME24	AIME25	AMC23	MATH500	Olympiad	Avg
SOTA Models							
o1	✗	74.3	**79.2**	-	96.4	-	-
DeepSeek-R1	✗	**79.8**	70.0	-	**97.3**	-	-
QwQ-32B	✗	79.5	65.3	**94.3**	92.3	**79.7**	**82.2**
Frontier Models (32B)							
DeepSeek-R1-32B	✗	72.9	59.0	88.8	94.3	72.5	77.5
START-32B	✓	66.7	47.1	**95.0**	94.4	-	-
STILL-3-TOOL-32B	✓	76.7	64.4	91.3	**96.6**	**75.9**	81.0
ReTool-R1-32B	✓	72.5	54.3	92.9	94.3	69.2	76.6
Prompt-Hint-SFT-32B	✓	**77.3**	65.0	**95.0**	**96.6**	75.1	**81.8**
Hint-Engineering-SFT-32B	✓	72.1	60.2	91.3	94.4	71.2	77.8
Hint-Engineering-RFT-32B	✓	76.7	**67.1**	94.4	95.1	73.4	81.3
Lightweight Models (1.5B)							
DeepSeek-R1-1.5B	✗	28.8	21.8	62.9	83.9	43.3	48.1
DeepScaleR-1.5B-Preview	✗	40.0	30.0	73.6	**87.8**	50.0	56.3
ToRL-1.5B	✓	26.7	26.7	67.5	77.8	44.0	48.5
Prompt-Hint-1.5B-SFT	✓	30.6	25.0	63.1	83.3	50.4	50.5
Prompt-Hint-1.5B-RL	✓	**43.1**	**30.2**	**73.8**	87.3	**57.1**	**58.3**
Hint-Engineering-1.5B-SFT	✓	34.0	23.5	64.6	84.2	49.8	51.2
Hint-Engineering-1.5B-RL	✓	41.0	29.4	70.0	85.8	55.6	56.4

Frontier models (32B). Among 32B tool-integrated models, Prompt-Hint-SFT-32B achieves the best average performance of 81.8%, surpassing all other tool-augmented baselines including STILL-3-TOOL-32B (81.0%) and ReTool-R1-32B (76.6%). Notably, Prompt-Hint-SFT-32B attains 77.3% on AIME24 and 65.0% on AIME25, outperforming the base DeepSeek-R1-32B by +4.4 and +6.0 points respectively. Hint-Engineering-RFT-32B provides the second-best average at 81.3%, with a particularly strong AIME25 score of 67.1%—the highest among all 32B models. Both CoRT variants substantially close the gap to full-scale SOTA models: Prompt-Hint-SFT-32B trails QwQ-32B by only 0.4 points in average, while operating with significantly fewer tokens (Figure 3).

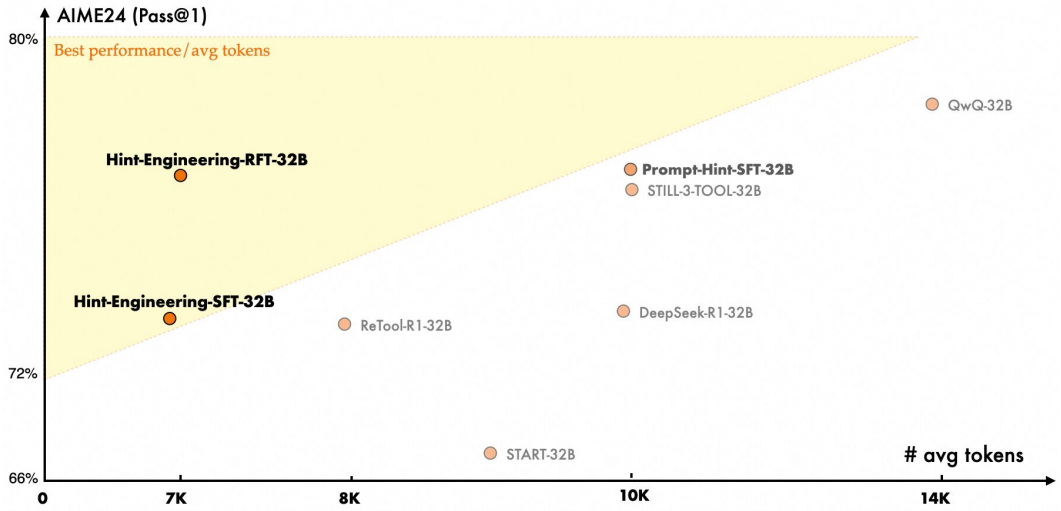
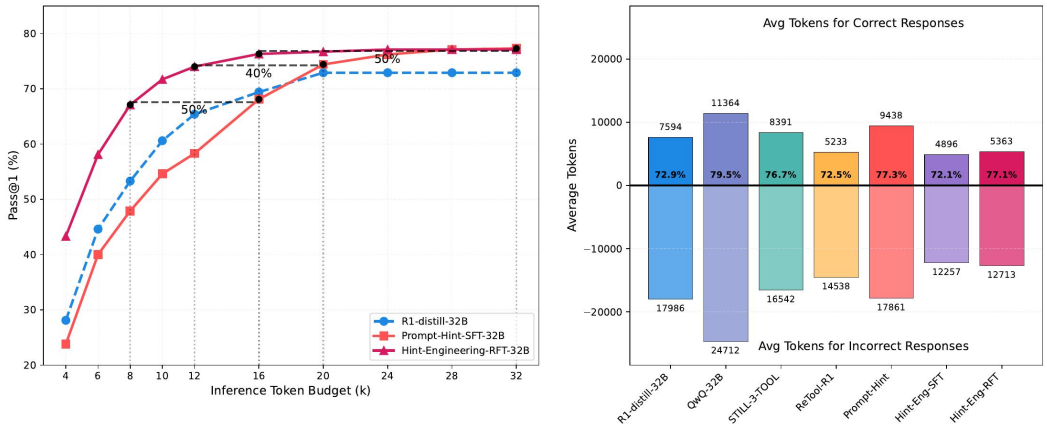


Figure 3: Performance vs. token efficiency comparison across models. CoRT variants achieve competitive accuracy with substantially fewer tokens.



(a) Performance on AIME24 with different token budget

(b) Average token usage on AIME24

Figure 4: Token efficiency analysis on AIME24. CoRT models achieve comparable or better accuracy with significantly fewer reasoning tokens.

Lightweight models (1.5B). The gains from CoRT are even more pronounced in the 1.5B setting. Prompt-Hint-1.5B-RL achieves the best overall average of 58.3%, representing a +10.2 point improvement over the base DeepSeek-R1-1.5B and surpassing DeepScaleR-1.5B-Preview (56.3%) which uses substantially more training data. On AIME24, Prompt-Hint-1.5B-RL reaches 43.1%, a +14.3 point gain over the base model. Hint-Engineering-1.5B-RL similarly achieves strong results with 56.4% average. Crucially, RL training provides large improvements over SFT alone: Prompt-Hint-1.5B-RL improves by +7.8 points average over its SFT counterpart, confirming that reinforcement learning effectively teaches smaller models to leverage code execution strategically. In contrast, ToRL-1.5B [8], which also applies RL for tool use, achieves only 48.5% average, suggesting that CoRT's hint-based approach provides a more effective learning signal for code integration.

4.3 Analysis

4.3.1 Token Efficiency

A key advantage of code-integrated reasoning is computational efficiency. Figure 3 and Figure 4 illustrate the token efficiency of CoRT models. In the 32B setting, Hint-Engineering-RFT-32B uses

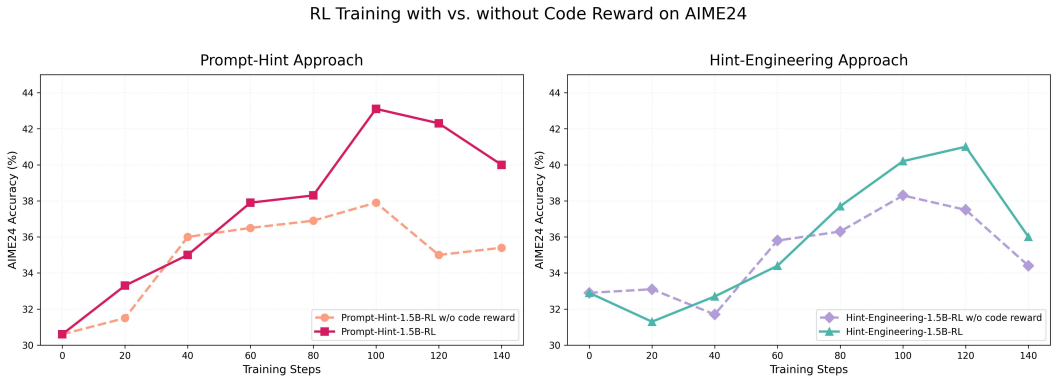


Figure 5: Ablation study on the impact of code execution reward (ω) during RL training. Adding a code reward with $\omega = 0.1$ provides consistent improvements on AIME24.

approximately 7K tokens on AIME24 compared to QwQ-32B's approximately 14K tokens—a roughly 50% reduction—while achieving comparable accuracy (76.7% vs. 79.5%). Across benchmarks, 32B CoRT models achieve an average token reduction of approximately 30% relative to non-tool baselines, while 1.5B models achieve even greater savings of approximately 50%. This efficiency gain stems from offloading precise numerical computation to the code interpreter, allowing the model to focus its token budget on high-level reasoning and problem decomposition rather than tedious arithmetic.

4.3.2 Ablation: Code Execution Reward

We ablate the effect of the code execution reward component R_c in our combined reward $R = R_a + \omega R_c$. As shown in Figure 5, incorporating the code reward with $\omega = 0.1$ provides up to +5% accuracy improvement on AIME24 during RL training. The code reward encourages the model to produce executable code and integrate interpreter outputs, preventing degenerate policies that avoid tool use entirely. Without this signal, models trained with RL occasionally regress to pure chain-of-thought reasoning, forgoing the computational advantages of code integration.

4.3.3 Pass@k Analysis

Figure 6 presents the pass@k scaling behavior of CoRT models on AIME24 and MATH500-Level5. Both prompt-hint and hint-engineering approaches exhibit strong pass@k scaling, indicating that the models produce diverse and complementary solution strategies across samples. This scaling property is practically valuable: it implies that CoRT models can benefit from majority voting or best-of-k selection at inference time, further boosting performance beyond the reported average results.

4.3.4 Code Usage Behavior

We analyze how CoRT training changes the model's code usage behavior. As shown in Figure 7, before training, the base model predominantly uses code for verification—checking intermediate results after deriving them through chain-of-thought. After CoRT training, the model shifts toward a more balanced pattern where code is used both for direct calculation and verification. This behavioral shift is significant: rather than redundantly re-checking results through code, the trained model proactively delegates computationally intensive steps to the interpreter, leading to more efficient and reliable problem solving. The shift is consistent across both the prompt-hint and hint-engineering variants, confirming that CoRT's training framework systematically improves code integration strategies.

5 Conclusion

We presented CoRT, a framework for integrating code execution within the reasoning process of large language models. CoRT addresses the challenge of teaching models when and how to invoke code interpreters by introducing targeted hint-engineering strategies combined with a lightweight

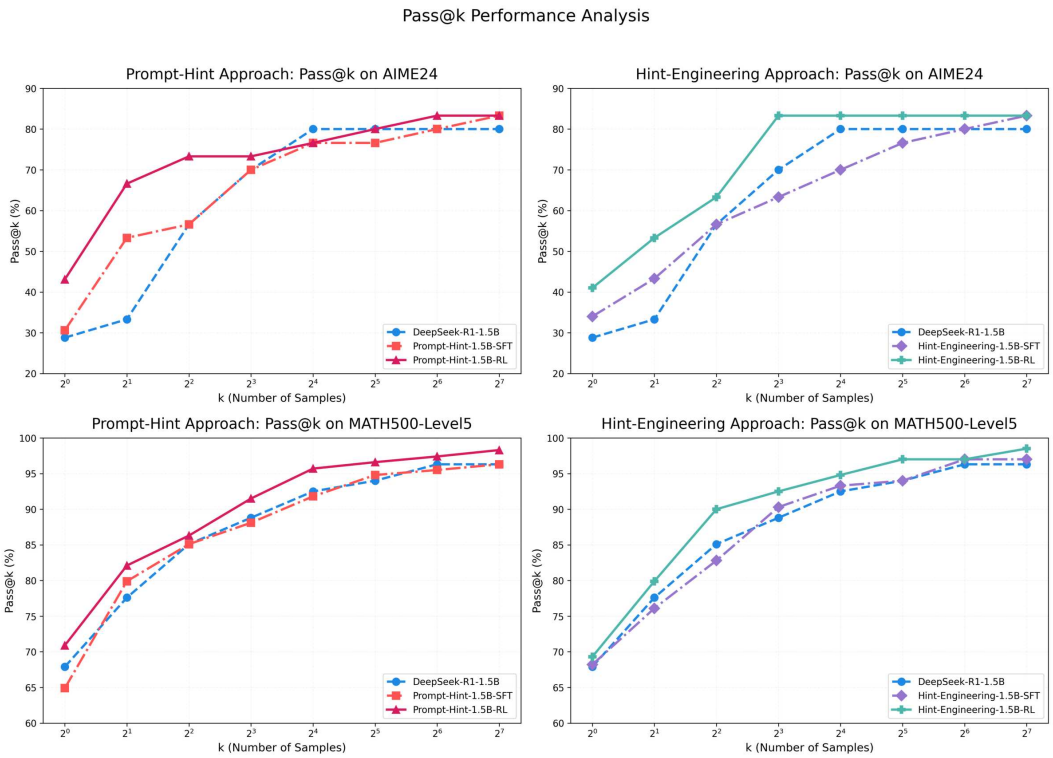


Figure 6: Pass@k performance on AIME24 (top) and MATH500-Level5 (bottom) for Prompt-Hint (left) and Hint-Engineering (right) approaches.

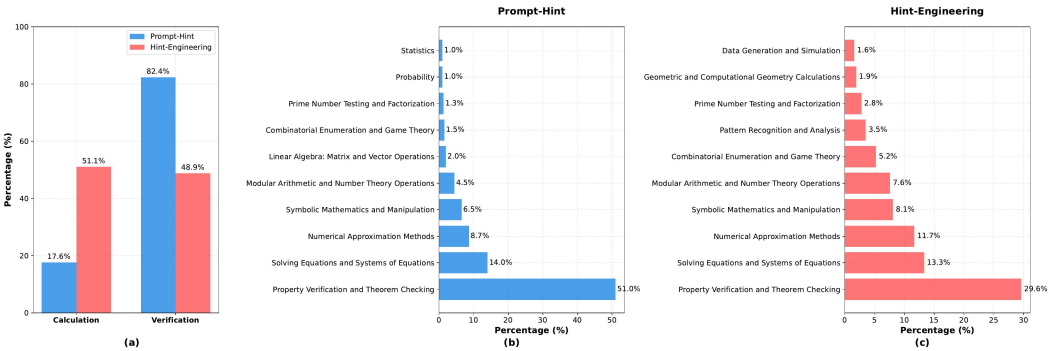


Figure 7: Analysis of Python code usage patterns before and after CoRT training. Training shifts the model from verification-heavy code use toward a balanced mix of calculation and verification.

post-training pipeline spanning SFT, RFT, and RL. Using only 30 curated exemplar traces, CoRT produces high-quality training data that enables effective code-integrated reasoning.

Our experiments across five mathematical reasoning benchmarks demonstrate that CoRT achieves strong results at both 32B and 1.5B scales. In the 32B setting, Prompt-Hint-SFT-32B achieves 81.8% average accuracy, approaching QwQ-32B performance with substantially fewer tokens. In the 1.5B setting, Prompt-Hint-1.5B-RL reaches 58.3% average accuracy, surpassing prior methods including DeepScaleR-1.5B-Preview. CoRT models consistently achieve 30–50% token reductions while maintaining competitive accuracy, demonstrating that strategic code integration offers a practical path toward more efficient mathematical reasoning. Our analysis reveals that CoRT training induces a qualitative shift in code usage behavior, moving from verification-heavy patterns to balanced calculation-verification strategies that better leverage the complementary strengths of neural reasoning and deterministic computation.

References

[1] OpenAI. Learning to reason with llms. *https://openai.com/index/learnin g-to-reason-with-llms/*, 2024.

[2] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025.

[3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[4] Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens, Niki van Stein, and Thomas Back. Reasoning with large language models, a survey. *arXiv preprint arXiv:2407.11511*, 2024.

[5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.

[6] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

[7] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*, 2023.

[8] Kezhou Wang, Ruijie Wu, Qinlin Zeng, Huao Lu, Hanye Wu, Qingfeng Cui, Haichao Lin, Yujia Liu, Xiaoyan Huang, Qingpeng Guo, Songtao Jian, Kaiyuan Lu, Shiyu Li, Hao Tian, Yongqin Sun, Xue Yang, Libin Song, Zejun Ou, and Guoqing Wang. ToRL: Scaling tool-integrated RL for LLMs. *arXiv preprint arXiv:2312.10372*, 2023.

[9] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms, 2025. URL https://arxiv.org/abs/2504.11536.

[10] Chengpeng Li, Mingfeng Xue, Zhenru Zhang, Jiaxi Yang, Beichen Zhang, Xiang Wang, Bowen Yu, Binyuan Hui, Junyang Lin, and Dayiheng Liu. Start: Self-taught reasoner with tools. *arXiv preprint arXiv:2503.04625*, 2025.

[11] Zhipeng Chen, Yingqian Min, Beichen Zhang, Jie Chen, Jinhao Jiang, Daixuan Cheng, Wayne Xin Zhao, Zheng Liu, Xu Miao, Yang Lu, et al. An empirical study on eliciting and improving r1-like reasoning models. *arXiv preprint arXiv:2503.04548*, 2025.

[12] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models, 2023. URL https://arxiv.org/abs/2308.01825.

[13] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[14] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024.

[15] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL https://qwenlm.github.io/blog/qwq-32b/.

[16] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2, 2025. Notion Blog.

[17] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025.

[18] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

[20] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 19(8):198343, 2025.

[21] Kefan Dong and Tengyu Ma. Beyond limited data: Self-play llm theorem provers with iterative conjecturing and proving. *arXiv preprint arXiv:2502.00212*, 2025.

[22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[23] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL https://arxiv.org/abs/2503.14476.

[24] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

[25] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning, 2025. URL https://arxiv.org/abs/2502.03387.

[26] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025. URL https://arxiv.org/abs/2501.19393.